

The eight core tables and how they relate

36 tables in total. These eight carry the business. What matters is not their columns — it's the arrows between them, because arrows are what you cannot change later.

THE PRINCIPLE THIS DOCUMENT IS ORGANISED AROUND

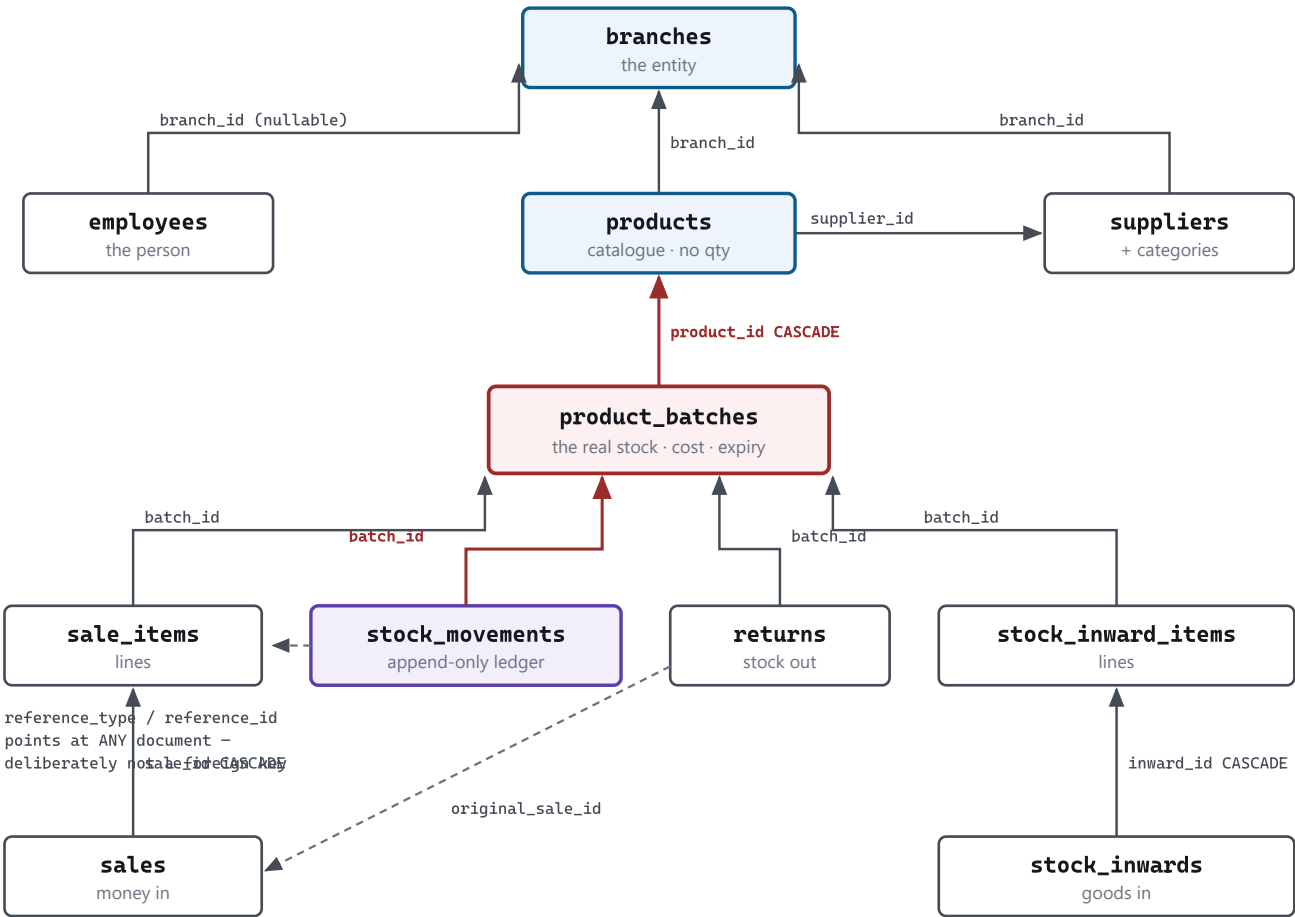
Columns are cheap to change. Relationships are not.

Adding `first_name` / `last_name` beside `name` costs nothing — no foreign key points at a column. But once `product_batches`, `sale_items` and `stock_movements` all point at `products.id` and there is live data on those links, changing *what points at what* means migrating every dependent row.

So every table below is described by its **keys and its arrows first**, and its columns second.

The map

—— foreign key (child → parent) - - - - - soft reference — no FK constraint **CASCADE** = deleting the parent deletes these rows



Every arrow is a real foreign key unless dashed. `branch_id` also exists on `sale_items` parents, `returns`, `stock_movements` and `stock_inwards` — omitted here to keep the picture readable.

READ THE MAP IN ONE SENTENCE

`branches` owns everything. `products` is a catalogue with no quantity. `product_batches` is the hub — it holds the actual stock, and four different things point at it. `stock_movements` records every change to it. `sales` and `stock_inwards` are documents that own their line rows.

Why this structure is safe to build on

1. Every primary key is a surrogate UUID

```
id uuid primary key default gen_random_uuid()
```

On all 36 tables, without exception. This is the single most important structural property of the schema, because it means **no business value is load-bearing in any relationship**.

`sku`, `name`, `invoice_number`, `batch_number`, `reference` — every one of them can be renamed, reformatted, split into two columns, or have its uniqueness rules changed, and not one foreign key notices. The arrows point at meaningless UUIDs, and meaningless UUIDs never need to change.

2. Both kinds of change have already been done, correctly

Migration `0006` is the evidence.

The cheap kind — six columns onto the most connected table in the schema, one statement, no data risk:

```
alter table products add column manufacturer text, add column image_url text, add column pricing_method text not null default 'fixed', add column margin_pe
```

The hard kind — `sku` went from globally unique to unique per entity, which meant adding a mandatory foreign key to a populated table:

```
alter table products add column branch_id uuid references branches(id); -- 1. nullable -- 2. backfill every existing row to an entity alter table products
```

THE RECIPE

Add nullable → backfill → set NOT NULL → swap the constraint. Four steps, no downtime, no orphans. That is how a structural change is done on live data, and it is already in the repository at `0006:111-172`.

3. One deliberate exception: the ledger's reference is soft

```
reference_type text, -- 'sale' | 'stock_inward' | 'return' | 'opening_stock' | ... reference_id uuid, -- no references clause reference_number text
```

`stock_movements` must point back at **six different document types**, and a single column cannot carry a foreign key to six different tables. So this link is polymorphic and unenforced by the database.

The trade-off, stated plainly: flexibility in exchange for no referential integrity on that one link. It is contained — only the `exp_*` functions ever write it, and they always write both halves together — but it is the one arrow in the system that Postgres does not guarantee.

Where a future change would actually hurt

EXPOSURE 1 · RETURNS AND STOCK_TRANSFERS HAVE NO LINE TABLES

Both carry `product_id`, `batch_id` and `quantity` **on the header row**. So one return is one product, one batch. Every other document in the system is header + lines; these two are not.

If a client asks for “a customer returned three different medicines on one slip”, that needs a new `return_items` table plus a migration of existing single-line returns into it. **That is a relationship change, not a column change** — the expensive category.

Not wrong today. It is simply the most likely structural request to arrive.

EXPOSURE 2 · THE CATALOGUE IS PER-ENTITY — THE BIGGEST BET IN THE SCHEMA

`products.branch_id` is `NOT NULL`. Each pharmacy owns its own product rows; ten entities selling paracetamol means ten `products` rows.

This is the right call for independently operated pharmacies and it is what makes the isolation model clean. But if a client later wants a **shared central catalogue with per-entity pricing on top**, that means splitting `products` into a shared master plus a per-entity overlay, and re-pointing `product_batches`, `sale_items`, `stock_movements` and six other tables.

If asked “what would be hardest to change?” — **this is the honest answer.**

One small thing to tidy

`product_batches.branch_id` and `sales.branch_id` were declared `on delete set null` in `0001`, then made `NOT NULL` in `0006`. Net effect: deleting a branch raises a constraint error rather than nulling the column. The data is safe, but the behaviour is an accident rather than a decision, and the error message would be confusing. `on delete restrict` would express the same rule on purpose. Two-line fix; not urgent, since `branches.is_active` is the intended path anyway.

The eight tables

Each card lists what the table points at, what points at it, and then — second — the columns that matter.

branches

ENTITY

the isolation boundary

One row = one independently operated pharmacy. The UI calls it an *Entity*.

PK	id uuid
Points at	nothing – it is the root
Pointed at by	every entity-owned table (20+) via branch_id

Columns that matter: code (UNIQUE, e.g. ENT-001), name, currency (default TZS), timezone, is_active.

LOCKED

branch_id NOT NULL on 20+ tables. Entity ownership is the deepest assumption in the schema — every RLS policy is built on it.

employees

IDENTITY

the person, and what they may do

Business identity, separate from the login. Supabase Auth holds the password; this table holds the role and permissions.

PK	id uuid
Points at	auth.users (auth_user_id, UNIQUE) · branches (branch_id, nullable – NULL only for master admins)
Pointed at by	sales.cashier_id · stock_movements.created_by · returns.requested_by / approved_by · stock_inwards.created_by / confirmed_by · audit_logs · employee_entities · and ~10 more created_by -style links

Columns that matter: role (4 values) · permission_overrides jsonb · max_discount_percent (per-person ceiling) · status.

CHEAP

New roles, new permissions, new per-person limits — all data or column changes. Two triggers guard this table; see the security reference.

products

CATALOGUE

what you sell — never how much you have

No quantity column, deliberately. This is a catalogue: name, pack size, price, reorder rules.

PK	id uuid
Points at	branches (branch_id NOT NULL) · categories · suppliers (default supplier)
Pointed at by	product_batches (CASCADE) · sale_items · stock_movements · returns · stock_inward_items · opening_stock_items · purchase_order_items · received_order_items · product_price_history · stock_transfers · stock_count_items – 11 tables

Columns that matter: sku (UNIQUE per entity, not global) · buy_price (SELECT revoked) · sell_price · pricing_method · margin_percent · max_discount_percent (per-product ceiling) · reorder_level / restock_target.

LOCKED

The most referenced table in the schema — 11 tables point at products.id. Adding columns is free; changing what a product is (see Exposure 2) is the expensive change.

One row = one physical lot, with its own cost, its own expiry and its own quantity. Two boxes of the same medicine bought at different prices are genuinely different rows.

PK	id uuid
Points at	products (product_id NOT NULL , ON DELETE CASCADE) · branches · suppliers
Pointed at by	stock_movements · sale_items · returns · stock_inward_items · opening_stock_items · received_order_items · stock_transfers - 7 tables

Columns that matter: batch_number · quantity_received · quantity_available (UPDATE revoked — ledger-derived) · unit_cost (SELECT revoked) · expiry_date · status · source_type / source_id .

LOCKED

This is the pivot of the whole design. source_type / source_id traces every batch back to the document that created it — the traceability regulators ask for. ON DELETE CASCADE from products means deleting a product destroys its stock history; in practice products are discontinued , never deleted.

stock_movements

LEDGER

append-only truth

Every stock change, ever, with the running balance. Nothing is updated or deleted. **Nothing points at this table** — it is terminal, which is what lets it be append-only.

PK	id uuid
Points at	product_batches · products · branches · employees (created_by) · <i>plus the soft</i> reference_type / reference_id pair
Pointed at by	nothing

Columns that matter: movement_type (13 values) · quantity_delta (signed) · balance_after · reason .

LOCKED

INSERT, UPDATE and DELETE are revoked from authenticated and anon . The single writer is fn_post_movement , revoked from every client role. The ledger sums exactly to product_batches.quantity_available , and a test asserts it.

sales → sale_items

DOCUMENT

money in

Header + lines. The header holds who, when, how paid and the totals; the lines hold which batch of which product, at what price.

PK

sales.id · sale_items.id

Points at

sales → customers , employees (cashier_id), branches

sale_items → sales (CASCADE), products , product_batches

Pointed at by

sale_items.sale_id · returns.original_sale_id

Columns that matter:

invoice_number (UNIQUE) · payment_method (5 values incl. M-Pesa, Selcom) · subtotal / discount / tax / total — all derived server-side · status · request_key (UNIQUE per entity — the double-click guard).

LOCKED

sale_items has no UPDATE or DELETE policy — immutable. Cancelling never deletes; it posts sale_reversal ledger rows and stamps who, when and why.

stock_inwards

→

stock_inward_items

DOCUMENT

goods in

Header + lines, draft until confirmed. Confirming creates the batches and writes the ledger in one transaction.

PK

stock_inwards.id · stock_inward_items.id

Points at

stock_inwards → suppliers , branches , employees , returns (supplier_return_id , links a replacement to its return)

stock_inward_items → stock_inwards (CASCADE) , products , product_batches (filled on confirm)

Pointed at by

stock_inward_items.inward_id

Columns that matter:

reference (UNIQUE) · inward_type (4 values) · invoice_number (UNIQUE per branch+supplier while not cancelled) · status · lines

carry both quantity and free_quantity .

NOTABLE

Paid units post as purchase ; free units post as a separate foc ledger movement. Free goods stay identifiable forever — most systems cannot tell you how much margin came from free stock.

One table serves customer returns, supplier returns, write-offs, expiry and staff consumption. They share one shape: stock moves, with a documented reason and a named approver.

PK	id uuid
Points at	products · product_batches · branches · employees (requested_by , approved_by , consumed_by) · sales (original_sale_id) · purchase_orders (original_po_id)
Pointed at by	stock_inwards.supplier_return_id

Columns that matter: reference (UNIQUE) · type (customer | supplier | damaged | expired | employee_consumption) · quantity · status · resolution_type · evidence_url .

THE EXPOSURE

product_id , batch_id and quantity sit on the header — **one return = one product, one batch**. A multi-line return slip would need a new return_items table and a data migration. See Exposure 1.

IF YOU TAKE ONE THING FROM THIS DOCUMENT

Every primary key is a meaningless UUID, so **no column is load-bearing in any relationship** — names, SKUs and invoice numbers can all change freely. The arrows are what's fixed, and they encode two real bets: that a product belongs to exactly one entity, and that a return concerns exactly one batch. Those are the two things that would cost real money to change; everything else is a column.